Avalonia Framework Documentation for Development

Table of Contents

1. Introduction to Avalonia
2. Creating an Avalonia Project in VS Code
3. Project Structure
4. XAML Basics - Getting Something on Screen
5. Controls and Drawing
6. Canvas Positioning
7. Understanding Events
8. User Input Handling
9. Creating a Game Loop
10. Working with Images and Sprites
11. Simple Complete Example: Bouncing Ball
12. Common Pitfalls

Introduction to Avalonia

Avalonia is a cross-platform UI framework for .NET that allows you to build desktop applications. In this assignment, we use Avalonia for two main purposes:

1. **Drawing Engine**: Rendering shapes and sprites on a screen
2. **Input Handling**: Capturing keyboard and mouse input from the user

Avalonia applications are built by combining:

- **XAML files** (`.axaml`): XML-based definitions for what appears on screen
- **C# code-behind files** (`.axaml.cs`): Code that handles logic, events, and interaction
- **Models/Logic classes**: Business logic separate from UI (like your `Game` and `Entity` classes)

Why Avalonia?

- **Cross-platform**: Same code runs on Windows, Linux, macOS
- **XAML-based**: Separates UI definition from logic
- **Event-driven**: Perfect for interactive applications

Creating an Avalonia Project in VS Code

Prerequisites

You should have:

- .NET 10 installed
- VS Code with C# extension installed

Step 1: Install Avalonia Templates

Open the VS Code terminal and install the Avalonia templates:

```
dotnet new install Avalonia.Templates
```

Step 2: Create a New Project in VS Code

1. Open VS Code
2. Press `Ctrl+Shift+P` (or `Cmd+Shift+P` on Mac) to open the Command Palette
3. Search for and select "**.NET: New Project**"
4. Choose "**Avalonia App**" template (not MVVM)
5. Select a folder where you want the project
6. Enter your project name (e.g., `MyGame`)

VS Code will create the project and open it automatically.

Step 3: Run the Application

1. Press `F5` or go to **Run** → **Start Debugging**
2. A window appears - your Avalonia app is running!

Step 4: Simplify the Template

The template creates some extra files you probably don't need. Update the main files:

MainWindow.axaml:

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Width="800" Height="450"
        x:Class="MyGame.MainWindow"
        Title="My Game">
    <Canvas x:Name="GameCanvas" Background="White" />
</Window>
```

MainWindow.axaml.cs:

```
using Avalonia.Controls;

namespace MyGame;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }
}
```

Now you have a clean starting point. Run `dotnet run` again. You should see a blank white window.

Project Structure

A typical Avalonia (game) project has this layout:

```
MyGame/
├── App.axaml              # Don't modify
├── App.axaml.cs           # Don't modify
├── MainWindow.axaml       # Your game window UI
├── MainWindow.axaml.cs    # Your game code
├── Program.cs             # Don't modify
├── Game.cs                # Your game logic
├── MyGame.csproj          # Don't modify
├── Assets/                # Your images and sprites
└── Entities/              # Your game objects
```

Important: `Program.cs`, `App.axaml`, and `App.axaml.cs` handle application startup. You don't need to modify these files. They work as-is. Focus on `MainWindow.axaml`, `MainWindow.axaml.cs` and your own classes.

XAML Basics - Getting Something on Screen

XAML is a markup language for defining your window layout. Here's what a basic game window looks like:

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Width="800" Height="450"
        x:Class="MyGame.MainWindow"
        Title="My Game">
    <Canvas x:Name="GameCanvas" Background="White" />
</Window>
```

- `<Canvas x:Name="GameCanvas">` is where you'll draw everything (from your Code Behind)

- `x:Name="GameCanvas"` lets you access it from the code behind as `GameCanvas`

Controls and Drawing

What We Want to Achieve

We need to draw things on the canvas. Avalonia provides controls for this. For example:

- `Rectangle`: For rectangular shapes
- `Ellipse`: For circles and ovals
- `Image`: For pictures and sprites

We add these from our C# code, and they appear on the canvas.

Adding Shapes Dynamically

Here's how to draw a red rectangle:

```
using Avalonia.Controls;
using Avalonia.Controls.Shapes;
using Avalonia.Media;

namespace MyGame;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();

        // Create a rectangle
        var rect = new Rectangle
        {
            Fill = new SolidColorBrush(Colors.Red),
            Width = 50,
            Height = 50
        };

        // Position it on the canvas
        Canvas.SetLeft(rect, 100);    // X position
        Canvas.SetTop(rect, 150);     // Y position

        // Add it to the canvas to display it
        GameCanvas.Children.Add(rect);
    }
}
```

Why do it this way? Because game objects move and appear/disappear during gameplay. Creating them in C# instead of in the XAML gives you much more control in code.

Other Drawing Controls

```
// A yellow circle (60×60 creates a perfect circle)
var circle = new Ellipse
{
    Fill = new SolidColorBrush(Colors.Yellow),
    Width = 60,
    Height = 60
};
GameCanvas.Children.Add(circle);

// An image from your Assets folder displayed at 32×32 pixels
var image = new Image
{
    Source = new Bitmap(AssetLoader.Open(new Uri(
        "avares://MyGame/Assets/player.png"))),
    Width = 32,
    Height = 32
};
GameCanvas.Children.Add(image);
```

Canvas Positioning

What We Want to Achieve

When you put shapes on a canvas, you need to tell Avalonia *where* to put them. We use coordinates: X and Y positions. Understanding the coordinate system is essential for moving objects, collision detection, and game logic.

Coordinate System

The canvas uses a simple coordinate system:

```
(0, 0) ──────────────────────→ X increases
  │
  │
  │
  ↓
Y increases
```

- **(0, 0)** is the **top-left corner**
- **X** increases going **right**
- **Y** increases going **down**
- Position is measured in **pixels**
- The (x,y) coordinate refers to the top left point of the drawing control, not the center.

Example in an 800×450 window:

- Top-left: (0, 0)
- Top-right: (800, 0)
- Bottom-left: (0, 450)
- Bottom-right: (800, 450)
- Center: (400, 225)

Setting Position

Use `Canvas.SetLeft()` and `Canvas.SetTop()` to position controls:

```
var rect = new Rectangle { Width = 50, Height = 50, Fill = ... };

// Position it at X=200, Y=100
Canvas.SetLeft(rect, 200);
Canvas.SetTop(rect, 100);

GameCanvas.Children.Add(rect);
```

Getting Position from a Control

```
double x = Canvas.GetLeft(rect);
double y = Canvas.GetTop(rect);
```

Updating Position (Moving)

```
// Move the rectangle 10 pixels right
var currentX = Canvas.GetLeft(rect);
Canvas.SetLeft(rect, currentX + 10);
```

Bounds and Collision Detection

A **bound** is the rectangular area a shape occupies. It's defined by: X, Y, Width, Height.

```
using Avalonia;

// Create a Rect representing the shape's area
var rect = new Rect(x, y, width, height);

// Check if two shapes overlap
var shape1 = new Rect(100, 100, 50, 50);      // At (100,100), size 50×50
var shape2 = new Rect(120, 120, 50, 50);      // At (120,120), size 50×50

if (shape1.Intersects(shape2))
{
    // They overlap!
}
```

This is how you detect collisions.

Understanding Events

Events are a way (not specific to Avalonia) of saying "something happened." They follow the **Publisher-Subscriber** pattern:

- **Publisher**: The object that raises the event (e.g., a timer, keyboard, mouse)

- **Event**: A specific thing that happened (e.g., "key was pressed", "button clicked", "timer ticked")
- **Subscriber**: Your code that responds to when that event occurs

How Events Work

Think of it like a news broadcast:

1. Something happens (user presses a key)
2. Avalonia detects it and raises an event (announces the news)
3. Your code is listening and gets called (receives the news)
4. Your code handles it (responds to the news)

Subscribing to Events

Subscribe to an event with the `+=` operator:

```
// "When KeyDown event happens, call my OnKeyDown method"
KeyDown += OnKeyDown!;

// "When the timer ticks, call my OnTick method"
_timer.Tick += OnTick!;

// "When the canvas is clicked, call my OnMouseClick method"
GameCanvas.PointerPressed += OnMouseClick!;
```

The `!` at the end is the null-forgiving operator - it tells the compiler "I know this is safe."

Event Handler Signature

Event handlers must have a specific shape. Most Avalonia events follow this pattern:

```
private void HandlerName(object sender, EventArgs e)
{
    // sender: The object that raised the event (who sent the news)
    // e: Event-specific information (details about the news)
}
```

Examples:

```
// Keyboard event - e contains key information
private void OnKeyDown(object sender, KeyEventArgs e)
{
    var key = e.Key;  // What key was pressed?
}

// Mouse event - e contains position information
private void OnMouseClick(object sender, PointerPressedEventArgs e)
{
    var position = e.GetPosition(GameCanvas);  // Where was clicked?
}

// Timer event - e is just EventArgs
private void OnTick(object sender, EventArgs e)
{
    // No special information, just "timer ticked"
}
```

Creating Your Own Events

You can make your own classes raise events too. For example, your `Game` class could announce when the game ends:

```
public class Game
{
    // Declare an event
    public event EventHandler? GameOver;

    public void Update()
    {
        // ... game logic ...

        if (SomeGameOverCondition)
        {
            // Tell everyone listening that the game is over
            GameOver?.Invoke(this, EventArgs.Empty);
        }
    }
}
```

Then subscribe to it:

```
_game = new Game();
_game.GameOver += OnGameOver!;

private void OnGameOver(object sender, EventArgs e)
{
    MessageBox.Show("Game Over!");
}
```

Custom Event Data

Sometimes you want to pass extra information with your event. Create custom event args:

```
// Custom event args class
public class ScoreChangedEventArgs : EventArgs
{
    public int OldScore { get; set; }
    public int NewScore { get; set; }
    public int PointsAdded { get; set; }
}

// In your Game class
public class Game
{
    public event EventHandler<ScoreChangedEventArgs>? ScoreChanged;

    public void AddScore(int points)
    {
        int oldScore = Score;
        Score += points;

        // Notify subscribers with detailed information
        ScoreChanged?.Invoke(this, new ScoreChangedEventArgs
        {
            OldScore = oldScore,
            NewScore = Score,
            PointsAdded = points
        });
    }
}

// Subscribe
_game.ScoreChanged += (sender, args) =>
{
    Console.WriteLine($"Score changed: {args.OldScore} → {args.NewScore} (+{args.PointsAdded})"
};
```

Why events? They let different parts of your code communicate without tight coupling. Your `Game` class doesn't need to know about your UI; it just announces what happened, and the UI listens.

Creating a Game Loop

A static game isn't fun. We need things to **move and change over time**. The game loop continuously updates game logic and redraws everything several times per second (typically 30-60 FPS).

```
public partial class MainWindow : Window
{
    private DispatcherTimer _timer;
    private Game _game;

    public MainWindow()
    {
        InitializeComponent();

        _game = new Game(800, 450);

        // Create timer - set interval to achieve desired FPS
        // For 30 FPS: 1000 / 30 ≈ 33 milliseconds
        _timer = new DispatcherTimer
        {
            Interval = TimeSpan.FromMilliseconds(1000 / 30)
        };

        _timer.Tick += OnTick!;
        _timer.Start();
    }

    private void OnTick(object sender, EventArgs e)
    {
        // Update game logic
        _game.Update();

        // Redraw everything
        Draw();
    }

    private void Draw()
    {
        // Update all visuals on screen
    }
}
```

Moving Objects with Velocity

Instead of moving objects directly in response to input (which feels jerky), use **velocity** - a value that says how many pixels to move each frame:

```
public partial class MainWindow : Window
{
    private double _playerX = 100;
    private double _playerVelocityX = 0;  // Pixels to move per frame
    private Rectangle _playerControl;

    public MainWindow()
    {
        InitializeComponent();

        _playerControl = new Rectangle
        {
            Fill = new SolidColorBrush(Colors.Blue),
            Width = 50,
            Height = 50
        };
        GameCanvas.Children.Add(_playerControl);

        _timer = new DispatcherTimer { Interval = TimeSpan.FromMilliseconds(1000 / 30) };
        _timer.Tick += OnTick!;
        _timer.Start();

        KeyDown += OnKeyDown!;
        KeyUp += OnKeyUp!;
    }

    private void OnKeyDown(object sender, KeyEventArgs e)
    {
        if (e.Key == Key.Left)
            _playerVelocityX = -5;  // Set velocity when key pressed
        else if (e.Key == Key.Right)
            _playerVelocityX = 5;
    }

    private void OnKeyUp(object sender, KeyEventArgs e)
    {
        if (e.Key == Key.Left || e.Key == Key.Right)
            _playerVelocityX = 0;  // Clear velocity when key released
    }

    private void OnTick(object sender, EventArgs e)
    {
        // Apply velocity each frame
        _playerX += _playerVelocityX;

        // Keep in bounds
        _playerX = Math.Clamp(_playerX, 0, 750);

        // Update visual
        Canvas.SetLeft(_playerControl, _playerX);
    }
}
```

Key insight: This creates smooth movement because the position is updated every frame, not just when a key is pressed.

State-Based Input Pattern

A more robust approach tracks which keys are currently pressed, then the game loop uses that state:

```
public class Game
{
    public bool KeyLeftPressed { get; set; }
    public bool KeyRightPressed { get; set; }
    public double PlayerX { get; private set; } = 100;

    public void Update()
    {
        // Update based on key state (pressed or not)
        if (KeyLeftPressed)
            PlayerX -= 5;
        if (KeyRightPressed)
            PlayerX += 5;

        PlayerX = Math.Clamp(PlayerX, 0, 750);
    }
}

public partial class MainWindow : Window
{
    private Game _game;
    private Rectangle _playerVisual;

    public MainWindow()
    {
        InitializeComponent();

        _game = new Game();
        _playerVisual = new Rectangle { Fill = new SolidColorBrush(Colors.Blue), Width = 50, He
        GameCanvas.Children.Add(_playerVisual);

        KeyDown += OnKeyDown!;
        KeyUp += OnKeyUp!;

        _timer = new DispatcherTimer { Interval = TimeSpan.FromMilliseconds(1000 / 30) };
        _timer.Tick += OnTick!;
        _timer.Start();
    }

    private void OnKeyDown(object sender, KeyEventArgs e)
    {
        if (e.Key == Key.Left) _game.KeyLeftPressed = true;
        if (e.Key == Key.Right) _game.KeyRightPressed = true;
    }

    private void OnKeyUp(object sender, KeyEventArgs e)
    {
        if (e.Key == Key.Left) _game.KeyLeftPressed = false;
        if (e.Key == Key.Right) _game.KeyRightPressed = false;
    }

    private void OnTick(object sender, EventArgs e)
    {
        _game.Update();  // Game uses key state to move player
        Canvas.SetLeft(_playerVisual, _game.PlayerX);
    }
}
```

This pattern is cleaner: your `Game` class doesn't need to know about keyboard events - it just reads a boolean. Your `MainWindow` just tracks what's pressed.

Working with Images and Sprites

What We Want to Achieve

Shapes are fine, but real games use images - characters, enemies, backgrounds. We need to load PNG or other image files and display them on the canvas.

Organizing Assets

Create an `Assets` folder in your project:

```
MyGame/
├── MyGame.csproj
├── Assets/
│   ├── player.png
│   ├── enemy.png
│   └── background.png
└── ...
```

Register Assets in Your Project File

Edit `MyGame.csproj` to tell the build system to include your images:

```
<ItemGroup>
    <AvaloniaResource Include="Assets\**" />
</ItemGroup>
```

This makes all files in the `Assets` folder available at runtime.

Loading and Displaying Images

Use `AssetLoader` to load images:

```
using Avalonia.Platform;
using Avalonia.Media.Imaging;
using Avalonia.Controls;

var playerImage = new Image
{
    Source = new Bitmap(AssetLoader.Open(new Uri("avares://MyGame/Assets/player.png"))),
    Width = 40,
    Height = 40
};

Canvas.SetLeft(playerImage, 100);
Canvas.SetTop(playerImage, 200);
GameCanvas.Children.Add(playerImage);
```

If you need to rotate the image, you can do that as follows:

```
playerImage.RenderTransform = new RotateTransform(_rotation); //where _rotation is the rotation
```

Ensure the RenderTransformOrigin property of the sprite is set to `new RelativePoint(0.5, 0.5, RelativeUnit.Relative)` if you want rotation to occur around the sprite's center.

Storing Image References

If you'll move the image later, store the control in a variable/property:

```
public partial class MainWindow : Window
{
    private Image _playerImage;

    public MainWindow()
    {
        InitializeComponent();

        // Create and store the image
        _playerImage = new Image
        {
            Source = new Bitmap(AssetLoader.Open(new Uri("avares://MyGame/Assets/player.png")))
            Width = 40,
            Height = 40
        };
        GameCanvas.Children.Add(_playerImage);
    }

    private void OnTick(object sender, EventArgs e)
    {
        // Move the image
        Canvas.SetLeft(_playerImage, _playerX);
        Canvas.SetTop(_playerImage, _playerY);
    }
}
```

Sprite Classes (Organizing Code)

As your game grows, create a class to organize sprite data:

```
public class Player
{
    public double X { get; set; }
    public double Y { get; set; }
    public Image Visual { get; }

    public Player(double x, double y)
    {
        X = x;
        Y = y;
        Visual = new Image
        {
            Source = new Bitmap(AssetLoader.Open(
                new Uri("avares://MyGame/Assets/player.png"))),
            Width = 40,
            Height = 40
        };
    }

    public void Update()
    {
        // Update position
    }

    public void Draw()
    {
        Canvas.SetLeft(Visual, X);
        Canvas.SetTop(Visual, Y);
    }
}
```

Then use it:

```
public partial class MainWindow : Window
{
    private Player _player;

    public MainWindow()
    {
        InitializeComponent();

        _player = new Player(100, 100);
        GameCanvas.Children.Add(_player.Visual);

        // Game loop...
    }

    private void OnTick(object sender, EventArgs e)
    {
        _player.Update();
        _player.Draw();
    }
}
```

Simple Complete Example: Bouncing Ball

What We're Building

A simple game where a ball bounces around the screen. When you click, a new ball spawns at that position. The balls gradually slow down due to friction.

Game Class

```
using Avalonia.Controls.Shapes;
using Avalonia.Media;
using System;
using System.Collections.Generic;

namespace BouncingBalls;

public class Ball
{
    public double X { get; set; }
    public double Y { get; set; }
    public double VelocityX { get; set; }
    public double VelocityY { get; set; }
    public const double Radius = 15;
    public const double WindowWidth = 800;
    public const double WindowHeight = 450;

    // The visual representation of this ball
    public Ellipse Visual { get; }

    public Ball(double x, double y)
    {
        X = x;
        Y = y;
        VelocityX = (Random.Shared.NextDouble() - 0.5) * 10;
        VelocityY = (Random.Shared.NextDouble() - 0.5) * 10;

        // Create the visual for this ball
        Visual = new Ellipse
        {
            Fill = new SolidColorBrush(Colors.Red),
            Width = Radius * 2,
            Height = Radius * 2
        };
    }

    public void Update()
    {
        // Apply velocity
        X += VelocityX;
        Y += VelocityY;

        // Apply friction (slow down)
        VelocityX *= 0.99;
        VelocityY *= 0.99;

        // Bounce off walls
        if (X - Radius < 0 || X + Radius > WindowWidth)
            VelocityX *= -1;
        if (Y - Radius < 0 || Y + Radius > WindowHeight)
            VelocityY *= -1;

        // Keep in bounds
        X = Math.Clamp(X, Radius, WindowWidth - Radius);
        Y = Math.Clamp(Y, Radius, WindowHeight - Radius);
    }
}
```

```
public class BallGame
{
    public List<Ball> Balls { get; } = new();

    public void AddBall(double x, double y)
    {
        Balls.Add(new Ball(x, y));
    }

    public void Update()
    {
        foreach (var ball in Balls)
        {
            ball.Update();
        }
    }
}
```

XAML

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Width="800" Height="450"
        x:Class="BouncingBalls.MainWindow"
        Title="Bouncing Balls">
    <Canvas x:Name="GameCanvas" Background="Black" />
</Window>
```

Window Code-Behind

```
using Avalonia.Controls;
using Avalonia.Input;
using Avalonia.Threading;
using System;

namespace BouncingBalls;

public partial class MainWindow : Window
{
    private BallGame _game;
    private DispatcherTimer _timer;

    public MainWindow()
    {
        InitializeComponent();

        _game = new BallGame();

        // Set up game loop (30 FPS)
        _timer = new DispatcherTimer { Interval = TimeSpan.FromMilliseconds(1000 / 30) };
        _timer.Tick += OnTick!;
        _timer.Start();

        // Handle mouse clicks
        GameCanvas.PointerPressed += OnMouseClick!;
    }

    private void OnMouseClick(object sender, PointerPressedEventArgs e)
    {
        var position = e.GetPosition(GameCanvas);
        var newBall = new Ball(position.X, position.Y);
        _game.AddBall(position.X, position.Y);
        GameCanvas.Children.Add(newBall.Visual);  // Add the ball's visual
    }

    private void OnTick(object sender, EventArgs e)
    {
        // Update game logic
        _game.Update();

        // Update visuals
        UpdateDisplay();
    }

    private void UpdateDisplay()
    {
        // Update each ball's position on screen
        foreach (var ball in _game.Balls)
        {
            Canvas.SetLeft(ball.Visual, ball.X - Ball.Radius);
            Canvas.SetTop(ball.Visual, ball.Y - Ball.Radius);
        }
    }
}
```

How It Works

1. **Click on canvas**: `OnMouseClick` is called, creates a new ball at that position
2. **Game loop**: `OnTick` fires ~30 times/second
3. **Update**: `_game.Update()` moves all balls, applies friction, handles bouncing
4. **Draw**: `UpdateDisplay()` positions each ball's visual on the canvas
5. **Repeat**: Back to step 2

Try This

- Run the game and click several times
- Observe balls bouncing and slowing down (friction)
- Try clicking near edges to see bouncing behavior

Common Pitfalls

1. Forgetting InitializeComponent()

```
// Wrong: InitializeComponent not called
public MainWindow()
{
    _game = new Game();  // NullReferenceException - GameCanvas is null!
    GameCanvas.Children.Add(rect);
}

// Correct: InitializeComponent first
public MainWindow()
{
    InitializeComponent();
    _game = new Game();
    GameCanvas.Children.Add(rect);
}
```

Why? `InitializeComponent()` creates all the controls from XAML. If you skip it, `GameCanvas` doesn't exist yet.

2. Asset Path Errors

```
// Wrong: relative path
new Bitmap("Assets/player.png")  // FileNotFoundException

// Correct: use avares protocol
new Bitmap(AssetLoader.Open(new Uri("avares://MyGame/Assets/player.png")))
```

Why? At runtime, your app doesn't have a file system path. Use `avares://` to reference embedded resources.

3. Event Handler Signature Mismatch

```
// Wrong: signature doesn't match
_timer.Tick += () => OnTick();  // Compilation error

// Correct: matches EventHandler signature
_timer.Tick += OnTick!;
```

4. Updating Position Without Game Loop

```
// Wrong: position changes, but not redrawn
private void OnKeyDown(object sender, KeyEventArgs e)
{
    if (e.Key == Key.Left)
        _playerX -= 5;
}

// Correct: position changes in game loop, redrawn every frame
private void OnKeyDown(object sender, KeyEventArgs e)
{
    if (e.Key == Key.Left)
        _playerVelocityX = -5;
}

private void OnTick(object sender, EventArgs e)
{
    _playerX += _playerVelocityX;
    Canvas.SetLeft(_playerControl, _playerX);
}
```

5. Not Clearing/Updating Visuals

```
// Problem: objects pile up, movement looks wrong
private void OnTick(object sender, EventArgs e)
{
    _playerX += _playerVelocityX;
    // Forgot to update Canvas position!
    // Player moves in code but not on screen
}

// Solution: always update canvas position
private void OnTick(object sender, EventArgs e)
{
    _playerX += _playerVelocityX;
    Canvas.SetLeft(_playerControl, _playerX);  // Update visuals
}
```

Additional Resources

- **Official Avalonia Docs**: https://docs.avaloniaui.net/